

**CESS-UFF / SGIMP**

Avaliação prática do processo seletivo

# **Sistema de sinalização com ESP32, LEDs, botão e OLED SSD1306**

Relatório técnico resumido

Candidato: [Sergio Cordero Calvimontes](#)  
Plataforma de simulação: Wokwi  
Linguagem: MicroPython  
Placa-alvo: Espressif ESP32-DevKitC V4

5 de agosto de 2026

# Sumário

|          |                                                      |          |
|----------|------------------------------------------------------|----------|
| <b>1</b> | <b>Introdução</b>                                    | <b>2</b> |
| <b>2</b> | <b>Requisitos e definições do projeto</b>            | <b>2</b> |
| 2.1      | Requisitos funcionais e estados do sistema . . . . . | 2        |
| 2.2      | Definições de hardware . . . . .                     | 2        |
| 2.3      | Entrada do botão e antirrepique . . . . .            | 3        |
| 2.4      | Interface do mostrador . . . . .                     | 4        |
| <b>3</b> | <b>Desenvolvimento do programa main.py</b>           | <b>4</b> |
| 3.1      | Estrutura geral . . . . .                            | 4        |
| 3.2      | Assincronismo cooperativo . . . . .                  | 4        |
| 3.3      | Controle do LED vermelho . . . . .                   | 5        |
| 3.4      | Controle do botão, LED verde e mensagens . . . . .   | 5        |
| 3.5      | Atualização dinâmica do OLED . . . . .               | 5        |
| <b>4</b> | <b>Estratégia de verificação e testes isolados</b>   | <b>6</b> |
| <b>5</b> | <b>Circuito simulado</b>                             | <b>6</b> |
| <b>6</b> | <b>Conclusão</b>                                     | <b>6</b> |
|          | <b>Referências eletrônicas</b>                       | <b>7</b> |

# 1 Introdução

Este projeto integra uma avaliação prática do processo seletivo **CESS-UFF / SGIMP**. A proposta reúne conteúdos de instrumentação, eletrônica e lógica de programação por meio da simulação de um sistema embarcado baseado em ESP32, dois diodos emissores de luz (LEDs), um botão pulsador – um tipo de interruptor que atua apenas enquanto está sendo acionado – e um mostrador (*display*) OLED controlado pelo circuito integrado SSD1306.

A simulação executável está publicada no Wokwi em <https://wokwi.com/projects/471528241540407297>. O código-fonte, o diagrama do circuito, a documentação e o histórico de versões estão disponíveis no repositório GitHub em <https://github.com/Argus-Kepheus/cess-uff>.

O Wokwi foi adotado como plataforma principal por oferecer suporte nativo ao ESP32 com MicroPython, edição visual do circuito, arquivo de conexão `diagram.json`, monitor serial e compartilhamento por endereço eletrônico. O repositório GitHub complementa a simulação ao fornecer rastreabilidade, controle de versões e uma estrutura adequada ao desenvolvimento no Visual Studio Code.

## 2 Requisitos e definições do projeto

### 2.1 Requisitos funcionais e estados do sistema

O sistema deve satisfazer os requisitos funcionais por meio de dois comportamentos independentes, executados concorrentemente sob escalonamento cooperativo: um controla exclusivamente o LED vermelho e o outro relaciona o estado do botão às saídas do LED verde e do mostrador OLED.

**Sinalização periódica do LED vermelho:** Alterna continuamente, independentemente do botão, do LED verde e do OLED, entre os estados **desligado** (GPIO 2 em nível baixo por aproximadamente 500 [ms]) e **ligado** (GPIO 2 em nível alto por aproximadamente 500 [ms]), compondo um ciclo completo de aproximadamente 1 [s].

**Interface com o usuário:** Lê o botão conectado ao GPIO 17 e atualiza o LED verde e o OLED: com o **botão solto**, GPIO 17 em nível baixo, LED verde (GPIO 4) desligado e OLED exibindo “Boa sorte!”; com o **botão pressionado**, GPIO 17 em nível alto, LED verde ligado e OLED exibindo “Conseguí”. O display é atualizado apenas na transição estável, evitando atualizações redundantes.

### 2.2 Definições de hardware

A placa selecionada foi a **Espressif ESP32-DevKitC V4**, representada no Wokwi pelo componente `board-esp32-devkit-c-v4`. A especificação original exige apenas “MicroPython para ESP32”; portanto, a escolha da placa não decorre de uma necessidade exclusiva de desempenho. Ela foi assumida por ser uma placa oficial da Espressif, documentada, identificada de forma inequívoca no simulador e capaz de disponibilizar todos os GPIOs requeridos. A Tabela 1 apresenta a pinagem específica adotada para este projeto.

Para eventual montagem física, recomenda-se uma ESP32-DevKitC V4 equipada com módulo ESP32-WROOM, preferencialmente ESP32-WROOM-32E. Versões com módulo WROVER devem ser evitadas sem revisão do projeto, pois GPIO 16 e GPIO 17 podem ser reservados para a memória PSRAM.

Além da placa selecionada, outras placas de desenvolvimento poderiam ser utilizadas sem problemas de compatibilidade de pinos, desde que baseadas no chip ESP32 (arquitetura Xtensa LX6) com módulo WROOM – e não WROVER, S2, S3, C3 ou C6, cujos mapeamentos de GPIO, reservas de memória e conjunto de periféricos diferem do exigido por este projeto. Entre as alternativas compatíveis estão:

- Espressif ESP32-DevKitC V2, a revisão anterior da placa adotada, com o mesmo módulo ESP32-WROOM-32 e cabeçalho (*header*) de 38 pinos;
- placas genéricas “NodeMCU-32S” (ESP-WROOM-32), amplamente disponíveis de diversos fabricantes, com layout de 38 pinos equivalente;
- “DOIT ESP32 DevKit V1”, com módulo ESP32-WROOM-32 em cabeçalho de 30 pinos, desde que os GPIOs 2, 4, 16, 17 e 25 estejam expostos;
- Clones com módulo WROOM-32 de fabricantes como AZ-Delivery ou HiLetgo, que replicam o mesmo mapeamento de GPIOs do módulo original.

Em todos os casos, é necessário confirmar que os GPIOs 16 e 17 não estão reservados para PSRAM ou outra função interna do módulo específico antes de reutilizar o mapeamento de pinos deste projeto.

Tabela 1: Pinagem específica do projeto.

| Função                         | GPIO | Terminal físico <sup>1</sup> | Identificador no Wokwi | Identificador no código |
|--------------------------------|------|------------------------------|------------------------|-------------------------|
| LED vermelho                   | 2    | J3-15                        | red-led                | red_led                 |
| LED verde                      | 4    | J3-13                        | green-led              | green_led               |
| Botão pulsador                 | 17   | J3-11                        | push-button            | push_button             |
| OLED, dados I <sup>2</sup> C   | 16   | J3-12                        | oled-display:SDA       | OLED_SDA_PIN            |
| OLED, relógio I <sup>2</sup> C | 25   | J2-9                         | oled-display:SCL       | OLED_SCL_PIN            |

<sup>1</sup>Identificação do conector (J2 ou J3) e da posição do terminal no cabeçalho físico de 38 pinos da ESP32-DevKitC V4, conforme docs/EN/hardware-reference.md.

A numeração utilizada corresponde aos números dos GPIOs do microcontrolador e não à posição sequencial dos terminais físicos da placa; a coluna “Terminal físico” indica exatamente essa posição sequencial, para referência ao montar o circuito fisicamente. Os dois LEDs possuem resistores limitadores de 220  $\Omega$ , e todos os componentes compartilham a mesma referência de terra.

## 2.3 Entrada do botão e antirrepique

O GPIO 17 é configurado como entrada com resistor interno de redução (*pull-down*). Dessa forma, o terminal permanece em nível baixo com o botão aberto e passa ao nível alto quando o contato conecta o GPIO à alimentação de 3,3 [V].

Essa configuração dispensa a implementação física de um resistor de *pull-down* externo: o próprio microcontrolador disponibiliza esse resistor internamente, ativado por software (`Pin.PULL_DOWN`), o que simplifica o circuito, reduz o número de componentes e elimina uma fonte de erro de montagem.

Não foi acrescentado filtro RC externo. O antirrepique (*bouncing*) é realizado por programa, sem bloqueio: o nível é amostrado periodicamente e uma alteração somente é aceita quando permanece estável por aproximadamente 30 [ms]. Essa estratégia é suficiente para a aplicação e evita oscilações do LED verde e atualizações repetidas do OLED.

Essas duas medidas fazem sentido sobretudo para uma implementação física: um botão pulsador mecânico real produz múltiplas transições elétricas espúrias durante o acionamento, e é esse ruído que o antirrepique elimina. O simulador Wokwi, por outro lado, modela o botão como um interruptor ideal, sem esse ruído de contato. Na simulação, portanto, o antirrepique não é estritamente necessário e é mantido apenas como boa prática, de modo que o mesmo código já esteja correto caso o projeto seja migrado para um botão físico sem qualquer alteração de lógica.

## 2.4 Interface do mostrador

O enunciado original exige apenas que o mostrador OLED exiba as mensagens conforme o estado do botão, sem especificar interface de comunicação ou pinos. A utilização da comunicação I<sup>2</sup>C, com GPIO 25 como SCL e GPIO 16 como SDA, foi definida antes da etapa de desenvolvimento, e não decorre de uma exigência explícita do enunciado. Uma vez definida, essa escolha foi tratada como fixa e declarada explicitamente no código, no diagrama e na documentação, para evitar ambiguidades ao longo do desenvolvimento.

A interface I<sup>2</sup>C utiliza apenas dois sinais de comunicação e é adequada às mensagens curtas e estáticas desta aplicação. Uma implementação SPI poderia alcançar maior taxa de transferência, porém exigiria mais sinais de controle e não proporcionaria vantagem relevante para as atualizações pouco frequentes deste projeto. O endereço adotado para o SSD1306 é 0x3C, com resolução de 128 por 64 [pixels].

## 3 Desenvolvimento do programa main.py

### 3.1 Estrutura geral

O arquivo `main.py` concentra a inicialização do hardware e a coordenação das tarefas.

Uma organização representativa da aplicação é mostrada a seguir:

Listing 1: Estrutura lógica resumida do programa.

```
1 async def blink_red_led():
2     """Toggle the red LED every 500 ms, independent of every other task."""
3     while True:
4         red_led.value(not red_led.value())
5         await asyncio.sleep_ms(RED_LED_BLINK_INTERVAL_MS)
6
7 async def monitor_button(initial_state):
8     """Sample, debounce, and apply stable button-state changes."""
9     ...
10
11 async def main():
12     initial_state = bool(push_button.value())
13     apply_button_state(initial_state)
14     asyncio.create_task(blink_red_led())
15     await monitor_button(initial_state)
```

Este trecho é uma síntese da arquitetura; o código completo e executável está disponível no repositório.

### 3.2 Assincronismo cooperativo

A biblioteca `asyncio` do MicroPython foi escolhida como arquitetura exclusiva do projeto. Em vez de um superlaço (*superloop*) que verifica manualmente diversos temporizadores, cada função independente pode ser organizada como uma corrotina (*coroutine*). A instrução `await asyncio.sleep_ms(...)` suspende apenas a tarefa atual e devolve o controle ao escalonador (*scheduler*).

Essa decisão foi tomada pelos seguintes motivos:

- Separação clara entre o piscar do LED vermelho, a leitura do botão e a atualização das saídas;
- Menor acoplamento entre funcionalidades e facilidade de manutenção e escalabilidade;

- Prevenção de congelamentos causados por `time.sleep()` no fluxo principal;
- Melhor tolerância às pequenas latências introduzidas pela transmissão por meio do barramento I<sup>2</sup>C.

O assincronismo adotado é cooperativo. Portanto, as tarefas devem alcançar pontos de espera com frequência e evitar rotinas longas ou bloqueantes.

### 3.3 Controle do LED vermelho

O LED vermelho opera em tarefa própria e alterna seu nível lógico a cada 500 [ms]. O ciclo completo, composto por 500 [ms] ligado e 500 [ms] desligado, dura aproximadamente um segundo. Essa tarefa não depende do botão, do LED verde ou do OLED.

### 3.4 Controle do botão, LED verde e mensagens

A tarefa de entrada realiza amostragem periódica, antirrepique e detecção de mudança de estado estável. Quando uma transição é confirmada, uma única rotina aplica o estado às duas saídas dependentes:

Tabela 2: Tabela de estados do botão, LED verde e OLED.

| Estado estável do botão | LED verde | Mensagem do OLED |
|-------------------------|-----------|------------------|
| Solto (LOW)             | Apagado   | Boa sorte!       |
| Pressionado (HIGH)      | Aceso     | Conseguí         |

A sincronização do LED verde e do OLED a partir do mesmo estado estável evita divergências entre a indicação luminosa e a mensagem apresentada.

### 3.5 Atualização dinâmica do OLED

O mostrador não é redesenhado continuamente. O quadro de imagem é transmitido na inicialização e somente quando o estado confirmado do botão muda. Essa atualização orientada por eventos reduz:

- Tráfego desnecessário no barramento I<sup>2</sup>C;
- Tempo de processamento;
- Repetição de escritas no controlador SSD1306;
- Cintilação (*flickering*) visual do mostrador;
- Interferência temporal nas demais tarefas.

A escolha é especialmente apropriada porque as mensagens permanecem estáticas entre uma transição e outra. Caso as mensagens não fossem estáticas – por exemplo, um contador ou uma leitura de sensor atualizada continuamente –, a estratégia adequada seria limitar a taxa de atualização do quadro a um intervalo fixo (dezenas ou centenas de milissegundos) em vez de redesenhar a cada iteração do laço, e, quando possível, retransmitir apenas a região alterada do *buffer* em vez do quadro completo, mantendo o tráfego I<sup>2</sup>C e o tempo de processamento dentro de limites aceitáveis.

## 4 Estratégia de verificação e testes isolados

O desenvolvimento foi acompanhado por testes progressivos. A separação dos subsistemas permitiu distinguir erros de ligação, inicialização e lógica de aplicação.

1. **Teste isolado do LED vermelho:** Utilizou um laço bloqueante mínimo somente para diagnóstico. O GPIO 2 foi alternado a cada 500 [ms] e o estado foi registrado no monitor serial. O objetivo foi confirmar placa, pino, resistor, polaridade e retorno ao terra sem interferência das demais funções.
2. **Teste do LED vermelho com `asyncio`:** Repetiu o teste anterior, agora encapsulado em uma corrotina agendada com `asyncio.create_task()` dentro de `asyncio.run()`, isolando especificamente o suporte a `asyncio` do firmware como variável, separado da fiação já confirmada.
3. **Teste isolado do botão e do LED verde:** Amostrou o GPIO 17 periodicamente e refletiu o estado lido no GPIO 4, sem qualquer dependência do LED vermelho ou do OLED. O objetivo foi confirmar a fiação do botão, o resistor interno de redução e o comportamento ativo em nível alto isoladamente.
4. **Teste básico do OLED:** Inicializou o barramento I<sup>2</sup>C em GPIO 25 e GPIO 16, confirmou a detecção do endereço 0x3C por meio de `i2c.scan()` e apresentou uma mensagem simples.
5. **Diagnóstico gráfico do SSD1306:** Exercitou todos os pixels ligados e desligados, padrões quadriculados complementares, varreduras horizontais e verticais, grade, pixels individuais, linhas, retângulos, áreas preenchidas, texto, inversão, contraste, desligamento, religamento e deslocamento do quadro.
6. **Integração final:** Após a validação individual, foram reunidos o piscar independente do LED vermelho, o LED verde comandado pelo botão e a atualização dinâmica das mensagens do OLED.

A sequência de testes reduziu o espaço de busca de falhas: se um subsistema simples não funcionasse, o problema poderia ser tratado antes da integração com o restante da aplicação.

## 5 Circuito simulado

A Figura 1 apresenta o estado atual do circuito no Wokwi. As cores dos condutores foram usadas para facilitar a inspeção visual: vermelho para alimentação e LED vermelho, preto para terra, verde para o LED verde, azul para o botão, amarelo para SCL e ciano para SDA.

## 6 Conclusão

O projeto atende aos requisitos propostos por meio de uma implementação modular e reproduzível. A escolha da ESP32-DevKitC V4 reduz ambiguidades de pinagem, enquanto o uso explícito dos GPIOs preserva a correspondência entre código, documentação e circuito. A comunicação I<sup>2</sup>C satisfaz adequadamente a necessidade de exibição das mensagens e mantém baixo o número de conexões.

A arquitetura assíncrona permite concorrência de processos. O antirrepique elimina a necessidade de filtro externo nesta aplicação, e a atualização do OLED apenas em mudanças de estado reduz processamento e cintilação. Por fim, os testes isolados e progressivos forneceram evidências de funcionamento de cada subsistema antes da integração.

A entrega é complementada por dois meios de acesso: a simulação pública no Wokwi e o repositório versionado no GitHub. Em conjunto, eles possibilitam executar, inspecionar, reproduzir e evoluir o projeto.

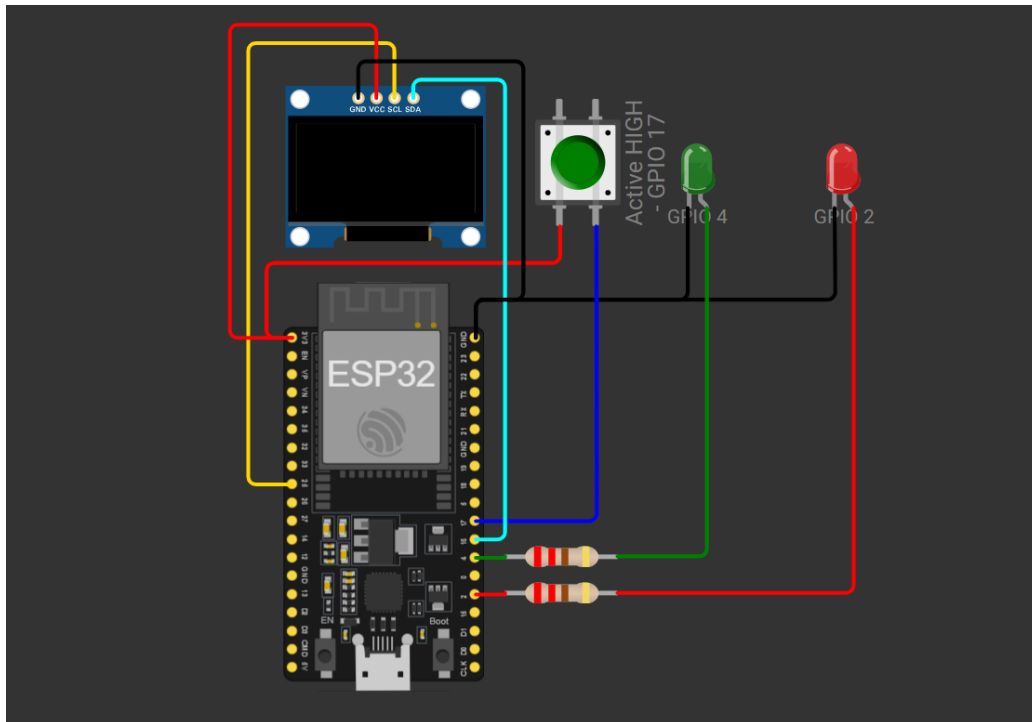


Figura 1: Circuito do projeto no Wokwi: ESP32-DevKitC V4, OLED SSD1306, botão e dois LEDs.

## Referências eletrônicas

- Projeto no Wokwi: <https://wokwi.com/projects/471528241540407297>
- Repositório no GitHub: <https://github.com/Argus-Kepheus/cess-uff>
- Guia da ESP32-DevKitC V4: [https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32/esp32-devkitc/user\\_guide.html](https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32/esp32-devkitc/user_guide.html)
- MicroPython para ESP32: <https://docs.micropython.org/en/latest/esp32/quickref.html>
- Documentação da asyncio: <https://docs.micropython.org/en/latest/library/asyncio.html>
- Componente SSD1306 do Wokwi: <https://docs.wokwi.com/parts/board-ssd1306>